

WEEK 03 MODULE
Programming Practice

TOPIC : 1 HEAP SORT

b) Medium

que. 2 **What is Sorting?**

- Sorting is the technique of arranging data elements in a particular order according to a chosen criterion, such as numerical value, alphabetical order, or date.

Types of Sorting :-

- i. Bubble Sort
- ii. Selection Sort
- iii. Insertion Sort
- iv. Merge Sort
- v. Quick Sort
- vi. Heap Sort

Heap Sort

Definition : Heap Sort is a comparison-based sorting algorithm that uses a Binary Heap to sort elements.

Idea:

1. Build a Max Heap from the given array (largest element becomes the root).
2. Swap the root (largest element) with the last element.
3. Reduce the heap size by one and restore the Max Heap.
4. Repeat until all elements are sorted.

Key Points:-

- Uses a Binary Max Heap.
- The largest element is always at the root.
- Repeatedly places the largest element at the end of the array.
- Uses the Heapify operation to maintain the heap property.
- In-place sorting algorithm.
- Not a stable sorting algorithm.
- Time Complexity: $O(n \log n)$ (Best, Average, and Worst Case).
- Space Complexity: $O(1)$.

How It Works:-

Array: 4 10 3 5 1

Step 1: Build Max Heap	Step 2: Swap Root with Last	Step 3: Swap Root with Last	Step 4: Swap Root with Last	Step 5: Swap Root with Last
<pre> 10 / \ 5 3 /\ 4 1</pre>	<pre> 1 / \ 5 3 / 4</pre> Sorted: 10 Heapify: <pre> 5 / \ 4 3 / 1</pre>	<pre> 1 / \ 4 3</pre> Sorted: 5 10 Heapify: <pre> 4 / \ 1 3</pre>	<pre> 3 / 1</pre> Sorted: 4 5 10	<pre>1</pre> Sorted: 3 4 5 10

Problem Statement

A multi-specialty hospital maintains records of emergency response times for all critical cases handled during a month. Each response time represents the number of minutes taken by the emergency team to reach and begin treatment for a patient.

Hospital administrators want to analyze the efficiency of their emergency services. To do so, all response times must first be sorted using **Heap Sort**.

After sorting the response times in ascending order, generate the following statistics:

1. Display the sorted response times.
2. Find the fastest response time.
3. Find the slowest response time.
4. Calculate the average response time.
5. Count the number of cases handled faster than the average.
6. Calculate the percentage of cases handled faster than the average.

The hospital intends to use this information to improve resource allocation and reduce emergency response delays.

1. Question Analysis

A multi-specialty hospital records the emergency response times for all critical cases handled during a month. These response times indicate the number of minutes taken by the emergency team to reach patients and begin treatment. Since the response times are stored in random order, they must first be arranged in ascending order to facilitate analysis.

After sorting, the hospital administrators need to determine the fastest response time, the slowest response time, the average response time, the number of cases handled faster than the average, and the percentage of such cases. As hospitals manage a large volume of emergency records, an efficient sorting algorithm is required. Therefore, the task is to implement the **Heap Sort** algorithm to sort the response times in ascending order without using any built-in sorting methods.

2. Problem Analysis

The given problem requires sorting an unsorted list of emergency response times in ascending order using the Heap Sort algorithm. Since the hospital maintains a large number of emergency records, the sorting technique should efficiently handle large datasets while providing consistent performance.

Heap Sort solves this problem by first constructing a Max Heap from the given response times. The largest element is repeatedly moved to the end of the array, and the heap is rebuilt until the entire array is sorted in ascending order. Once sorting is completed, the fastest response time, slowest response time, average response time, number of cases handled faster than the average, and the percentage of such cases are calculated to evaluate the efficiency of the hospital's emergency services.

3. Algorithm Selection Analysis

Selected Algorithm: Heap Sort

- Heap Sort is based on the Binary Heap data structure.
- It first constructs a Max Heap from the given response times.
- The maximum element is repeatedly placed at the end of the array.
- It guarantees a time complexity of $O(N \log N)$ in the best, average, and worst cases.
- It efficiently handles large datasets while requiring only constant extra memory.
- After sorting, the required statistical information can be calculated easily.

4. Points to Remember

- Heap Sort is based on the Max Heap data structure.
- A Max Heap stores the largest element at the root.
- The heap is built before the sorting process begins.
- During each iteration, the root element is swapped with the last element of the heap.

- The heap size decreases after every swap.
- The heap is rebuilt using the heapify() method after each extraction.
- The process continues until the complete array is sorted in ascending order.
- After sorting, the fastest response time is obtained from the first element.
- The slowest response time is obtained from the last element.
- The average response time is calculated using the total sum of all response times.
- The number of cases faster than the average is counted.
- The percentage of cases faster than the average is calculated.
- Time Complexity
 - Best Case : $O(N \log N)$
 - Average Case : $O(N \log N)$
 - Worst Case : $O(N \log N)$
- Space Complexity : $O(1)$

5. Core Idea / Logic

- Store all emergency response times in an array.
- Build a Max Heap from the given response times.
- Repeatedly swap the root element with the last element of the heap.
- Reduce the heap size after every swap.
- Apply the heapify() method to restore the Max Heap property.
- Continue the process until the entire array is sorted in ascending order.
- Display the sorted response times.
- Determine the fastest response time from the first element of the sorted array.
- Determine the slowest response time from the last element of the sorted array.
- Calculate the average response time.
- Count the number of cases whose response time is less than the average.
- Calculate the percentage of cases handled faster than the average.
- Display all the required statistical information.

6. Execution Flow

Part A : Execution Flow

Input Phase

1. The program starts execution from the main() method.
2. The user enters the number of emergency response times (N).
3. The program validates whether N is within the specified range.
4. An integer array of size N is created.
5. The user enters all emergency response times.
6. Each response time is validated and stored in the array.
7. The heapSort() method is called to sort the response times in ascending order.

Processing Phase

1. The heapSort() method first builds a **Max Heap** from the given response times.
2. The root element (largest response time) is swapped with the last element of the heap.
3. The heap size is reduced by one after every swap.
4. The heapify() method restores the Max Heap property for the remaining heap.
5. Steps 2–4 are repeated until the complete array becomes sorted in ascending order.
6. The fastest response time is obtained from the first element of the sorted array.
7. The slowest response time is obtained from the last element of the sorted array.
8. The average response time is calculated.
9. The number of cases handled faster than the average is counted.
10. The percentage of cases handled faster than the average is calculated.

Output Phase

1. Control returns to the main() method.
2. The sorted response times are displayed.
3. The fastest response time is displayed.
4. The slowest response time is displayed.
5. The average response time is displayed.
6. The number of cases handled faster than the average is displayed.
7. The percentage of cases handled faster than the average is displayed.
8. The program terminates successfully.

Part B : Steps

Step 1: Start the execution of the program.

Step 2: Read the number of emergency response times (N) from the user.

Step 3: Validate the value of N. If N is less than 1 or greater than 100000, display an error message and terminate the program.

Step 4: Create an integer array of size N to store the emergency response times.

Step 5: Read each emergency response time from the user, validate that every value lies between 1 and 500, store the valid values in the array, and simultaneously calculate the total sum of all response times.

Step 6: Call the heapSort(arr, n) method to sort the response times in ascending order.

Step 7: Build a **Max Heap** from the given response times by calling the heapify() method for all non-leaf nodes.

Step 8: Repeatedly swap the root element (largest response time) with the last element of the current heap.

Step 9: Reduce the heap size by one and call the heapify() method again to restore the Max Heap property.

Step 10: Repeat the extraction and heapify process until the complete array becomes sorted in ascending order.

Step 11: Display the sorted response times.

Step 12: Store the first element of the sorted array as the fastest response time and the last element as the slowest response time.

Step 13: Calculate the average response time by dividing the total sum of all response times by N.

Step 14: Traverse the sorted array and count the number of response times that are less than the average response time.

Step 15: Calculate the percentage of cases handled faster than the average using the formula:

$$\text{Percentage} = (\text{Count} \div N) \times 100$$

Step 16: Display the fastest response time, slowest response time, average response time, number of cases handled faster than the average, and the percentage of such cases.

Step 17: Close the Scanner object and terminate the program.

7. Program Structure Analysis

i. Variable n

int n;

Purpose:

- Stores the total number of emergency response times entered by the user.
 - Determines the size of the array.
 - Defines the total number of records to be sorted.
- Constraint:

$1 \leq n \leq 100000$

- Minimum value of n can be 1.
- Maximum value of n can be 100000.

ii. Array arr[]

int[] arr = new int[n];

Purpose:

- Stores all emergency response times.
 - Acts as the main array on which Heap Sort is performed.
 - Contains the response times in ascending order after sorting.
- Constraint:

$1 \leq \text{Time} \leq 500$

- Minimum response time can be 1 minute.
- Maximum response time can be 500 minutes.

iii. Variable totalSum

int totalSum = 0;

Purpose:

- Stores the sum of all emergency response times.
- Used to calculate the average response time.
- Is updated while reading the input values.

iv. Variable average

double average = (double) totalSum / n;

Purpose:

- Stores the average response time of all emergency cases.
- Acts as the reference value for determining whether a case was handled faster than average.

v. Variable count

```
int count = 0;
```

Purpose:

- Counts the number of emergency cases whose response time is less than the average response time.
- Increases whenever a response time satisfies the required condition.

vi. Variable percentage

```
double percentage = ((double) count / n) * 100;
```

Purpose:

- Stores the percentage of emergency cases handled faster than the average.
- Converts the count into a percentage using the total number of cases.

vii. Variable fastest

```
int fastest = arr[0];
```

Purpose:

- Stores the minimum response time after sorting.
- Represents the fastest emergency response.

viii. Variable slowest

```
int slowest = arr[n - 1];
```

Purpose:

- Stores the maximum response time after sorting.
- Represents the slowest emergency response.

ix. Variables largest, left and right

```
int largest = i;  
int left = 2 * i + 1;  
int right = 2 * i + 2;
```

Purpose:

- largest stores the index of the largest value among the parent and its children.
- left stores the index of the left child.
- right stores the index of the right child.
- These variables help maintain the Max Heap property during the heapify process.

x. Variable temp

```
int temp;
```

Purpose:

- Temporarily stores an element during swapping.
- Prevents data loss while exchanging two response times.

xi. Input Loop

```
for(int i = 0; i < n; i++)  
{  
    arr[i] = sc.nextInt();  
}
```

Purpose:

- Accepts all emergency response times from the user.
- Validates every response time according to the given constraints.
- Stores the values in the array.
- Simultaneously calculates the total sum of all response times.

xii. heapSort() Method

```
heapSort(arr, n);
```

Purpose:

- Performs Heap Sort on the array.
- Builds a Max Heap from the given response times.
- Repeatedly places the largest element at its correct position.
- Produces the final array in ascending order.

xiii. heapify() Method

```
heapify(arr, n, i);
```

Purpose:

- Maintains the Max Heap property.
- Compares the parent node with its left and right children.
- Swaps elements whenever a child is larger than the parent.
- Continues recursively until the subtree satisfies the Max Heap condition.

xiv. Heap Construction Loop

```
for(int i = n / 2 - 1; i >= 0; i--)
```

Purpose:

- Starts from the last non-leaf node.
- Calls the heapify() method for every non-leaf node.
- Builds the initial Max Heap before sorting begins.

xv. Heap Extraction Loop

```
for(int i = n - 1; i > 0; i--)
```

Purpose:

- Repeatedly swaps the root element with the last element of the current heap.
- Reduces the heap size after every iteration.
- Calls heapify() to restore the Max Heap property.
- Continues until the array becomes completely sorted.

xvi. Count Loop

```
for(int i = 0; i < n; i++)
```

Purpose:

- Traverses the sorted array.
- Compares each response time with the average response time.
- Counts the number of emergency cases handled faster than the average.

xvii. Output Loop

```
for(int i = 0; i < n; i++)  
{  
    System.out.print(arr[i] + " ");  
}
```

Purpose:

- Traverses the sorted array.
- Displays all emergency response times in ascending order.
- Produces the final sorted output before displaying the calculated statistics.

8. Java Program

```
package week3_module;
```

```
import java.util.Scanner;
```

```
public class HeapSortMedium {
```

```
    // Heapify function  
    static void heapify(int[] arr, int n, int i) {
```

```
        int largest = i;  
        int left = 2 * i + 1;  
        int right = 2 * i + 2;
```

```
        if (left < n && arr[left] > arr[largest]) {  
            largest = left;  
        }
```

```
        if (right < n && arr[right] > arr[largest]) {  
            largest = right;  
        }
```

```
        If (largest != i) {
```

```
            int temp = arr[i];  
            arr[i] = arr[largest];  
            arr[largest] = temp;
```

```
            heapify(arr, n, largest);
```

```
        }
```

```
    }
```

```
    // Heap Sort function  
    static void heapSort(int[] arr, int n) {
```



```

// Build Max Heap
for (int i = n / 2 - 1; i >= 0; i--) {
    heapify(arr, n, i);
}

// Extract elements one by one
for (int i = n - 1; i > 0; i--) {

    int temp = arr[0];
    arr[0] = arr[i];
    arr[i] = temp;

    heapify(arr, i, 0);
}

}

public static void main(String[] args) {

    Scanner sc = new Scanner(System.in);

    System.out.print("Enter the number of emergency cases : ");
    int n = sc.nextInt();

    if (n < 1 || n > 100000) {
        System.out.println("Error: N must be between 1 and 100000.");
        sc.close();
        return;
    }

    int[] arr = new int[n];
    int totalSum = 0;

    System.out.print("Enter the emergency response times (in minutes) : ");

    for (int i = 0; i < n; i++) {

        arr[i] = sc.nextInt();

        if (arr[i] < 1 || arr[i] > 500) {
            System.out.println("Error: Response time must be between 1 and 500.");
            sc.close();
            return;
        }

        totalSum += arr[i];
    }

    // Perform Heap Sort
    heapSort(arr, n);

    // Display sorted response times
    System.out.print("Sorted Response Times : ");
    for (int i = 0; i < n; i++) {
        System.out.print(arr[i] + " ");
    }
    System.out.println();
}

```

```

        // Fastest response time
        int fastest = arr[0];
        System.out.println("Fastest: " + fastest);

        // Slowest response time
        int slowest = arr[n - 1];
        System.out.println("Slowest: " + slowest);

        // Average response time
        double average = (double) totalSum / n;
        System.out.printf("Average: %.2f%n", average);

        // Count cases faster than average
        int count = 0;

        for (int i = 0; i < n; i++) {
            if (arr[i] < average) {
                count++;
            }
        }

        System.out.println("Cases Faster Than Average: " + count);

        // Percentage of cases faster than average
        double percentage = ((double) count / n) * 100;
        System.out.printf("Percentage: %.2f%%\n", percentage);

        sc.close();
    }
}

```

The sample output displays:

```

Average: 10.75
Percentage: 62.50%

```

Using:

System.out.printf("Average: %.2f%n", average);
ensures the average is always displayed with exactly two decimal places.
Similarly, System.out.printf("Percentage: %.2f%%\n", percentage);
prints the percentage with two decimal places followed by the % symbol, matching the required output format.

9. Program's output

```

Enter the number of emergency cases : 8
Enter the emergency response times (in minutes) : 12 7 15 9 20 5 10 8
Sorted Response Times : 5 7 8 9 10 12 15 20
Fastest: 5
Slowest: 20
Average: 10.75
Cases Faster Than Average: 5
Percentage: 62.50%

```

10. Screenshot

program

```
HeapSortMedium.java X
1 package week3_module;
2 import java.util.Scanner;
3 public class HeapSortMedium {
4
5     // Heapify function
6     static void heapify(int[] arr, int n, int i) {
7
8         int largest = i;
9         int left = 2 * i + 1;
10        int right = 2 * i + 2;
11
12        if (left < n && arr[left] > arr[largest]) {
13            largest = left;
14        }
15
16        if (right < n && arr[right] > arr[largest]) {
17            largest = right;
18        }
19
20        if (largest != i) {
21            int temp = arr[i];
22            arr[i] = arr[largest];
23            arr[largest] = temp;
24
25            heapify(arr, n, largest);
26        }
27    }
28
29    // Heap Sort function
30    static void heapSort(int[] arr, int n) {
31
32        // Build Max Heap
33        for (int i = n / 2 - 1; i >= 0; i--) {
34            heapify(arr, n, i);
35        }
36
37        // Extract elements one by one
38        for (int i = n - 1; i > 0; i--) {
39
40            int temp = arr[0];
41            arr[0] = arr[i];
42            arr[i] = temp;
43
44            heapify(arr, i, 0);
45        }
46    }
47
48    public static void main(String[] args) {
49
50        Scanner sc = new Scanner(System.in);
51
52        System.out.print("Enter the number of emergency cases : ");
53        int n = sc.nextInt();
54        if (n < 1 || n > 100000) {
55            System.out.println("Error: N must be between 1 and 100000.");
56            sc.close();
57            return;
58        }
59    }
```

```
60
61    int[] arr = new int[n];
62    int totalSum = 0;
63    System.out.print("Enter the emergency response times (in minutes) : ");
64    for (int i = 0; i < n; i++) {
65
66        arr[i] = sc.nextInt();
67
68        if (arr[i] < 1 || arr[i] > 500) {
69            System.out.println("Error: Response time must be between 1 and 500.");
70            sc.close();
71            return;
72        }
73
74        totalSum += arr[i];
75    }
76
77    // Perform Heap Sort
78    heapSort(arr, n);
79
80    // Display sorted response times
81    System.out.print("Sorted Response Times : ");
82    for (int i = 0; i < n; i++) {
83        System.out.print(arr[i] + " ");
84    }
85    System.out.println();
86}
```

```

87 // Fastest response time
88 int fastest = arr[0];
89 System.out.println("Fastest: " + fastest);
90
91 // Slowest response time
92 int slowest = arr[n - 1];
93 System.out.println("Slowest: " + slowest);
94
95 // Average response time
96 double average = (double) totalSum / n;
97 System.out.printf("Average: %.2f%n", average);
98
99 // Count cases faster than average
100 int count = 0;
101
102 for (int i = 0; i < n; i++) {
103     if (arr[i] < average) {
104         count++;
105     }
106 }
107
108 System.out.println("Cases Faster Than Average: " + count);
109
110 // Percentage of cases faster than average
111 double percentage = ((double) count / n) * 100;
112 System.out.printf("Percentage: %.2f%%n", percentage);
113
114 sc.close();
115 }
116 }
117

```

program's output

Problems @ Javadoc Declaration Console X

<terminated> HeapSortMedium [Java Application] /Library/Java/JavaVirtualMachin

```

Enter the number of emergency cases : 8
Enter the emergency response times (in minutes) : 12 7 15 9 20 5 10 8
Sorted Response Times : 5 7 8 9 10 12 15 20
Fastest: 5
Slowest: 20
Average: 10.75
Cases Faster Than Average: 5
Percentage: 62.50%

```